데이터베이스

Relation – 구조 (Structure)

오병우

컴퓨터공학부

kit 국립금오공과대학교
Kumoh National Institute of Technology

관계 데이터 모델

- 관계 데이터 모델(relational data model)의 탄생
  - 1970년 IBM의 Edgar Frank "Ted" Codd에 의해 제안
    - Data Structure
    - Data manipulation (Operation)
    - Data integrity (Constrain)
- 관계 데이터 모델의 특성
  - 릴레이션(relation)과 수학적인 이론에 기초
  - 일반 사용자는 테이블 형태로 생각
    - 통상적인 테이블의 개념과는 다름
      - 관계 데이터 모델의 직관적인 이해에는 도움이 됨
    - 테이블의 열(column) = 필드(field) 혹은 아이템(item)
      ≒ 관계 데이터 모델의 애트리뷰트(attribute)
    - 테이블의 행(row) = 레코드(record)
      ≒ 관계 데이터 모델의 투플(tuple)

Ted는 별명.

E.F. Codd

데이터 모델 정의 S.O.C

Information Retrieval P. BAXENDALE, Editor

A Relational Model of Data for Large Shared Data Banks

E. F. Codd
IBM Research Laboratory, San Jose, California

Future users of large data banks must be protected from having to know how the data is organized in the machine (the internal representation). A prompting service which supplies such information is not a satisfactory solution. Activities of users at terminals and most application programs should remain unaffected when the internal representation of data is changed and even when some aspects of the external representation are changed. Changes in data representation will often be needed as a result of changes in query, update, and report traffic and natural growth in the types of stored information.

Existing noninferential, formatted data systems provide users with tree-structured files or slightly more general network models of the data. In Section 1, inadequacies of these models are discussed. A model based on *n*-ary relations, a normal form for data base relations, and the concept of a universal data sublanguage are introduced. In Section 2, certain operations on relations (other than logical inference) are discussed and applied to the problems of redundancy and consistency in the user's model.

KEY WORDS AND PHRASES: data bank, data base, data structure, data organization, hierarchies of data, networks of data, relations, derivability, redundancy, consistency, composition, join, retrieval language, predicate calculus, security, data integrity
CR CATEGORIES: 3.70, 3.73, 3.75, 4.20, 4.22, 4.29

1. Relational Model and Normal Form

1.1. Introduction
This paper is concerned with the application of elementary relation theory to systems which provide shared access to large banks of formatted data. Except for a paper by Childs [1], the principal application of relations to data systems has been to deductive question-answering systems. Levein and Maron [2] provide numerous references to work in this area.

In contrast, the problems treated here are those of *data independence*—the independence of application programs and terminal activities from growth in data types and changes in data representation—and certain kinds of *data inconsistency* which are expected to become troublesome even in nondeductive systems.

The relational view (or model) of data described in Section 1 appears to be superior in several respects to the graph or network model [3, 4] presently in vogue for noninferential systems. It provides a means of describing data with its natural structure only—that is, without superimposing any additional structure for machine representation purposes. Accordingly, it provides a basis for a high level data language which will yield maximal independence between programs on the one hand and machine representation and organization of data on the other.

A further advantage of the relational view is that it forms a sound basis for treating derivability, redundancy, and consistency of relations—these are discussed in Section 2. The network model, on the other hand, has spawned a number of confusions, not the least of which is mistaking the derivation of connections for the derivation of relations (see remarks in Section 2 on the "connection trap").

Finally, the relational view permits a clearer evaluation of the scope and logical limitations of present formatted data systems, and also the relative merits (from a logical standpoint) of competing representations of data within a single system. Examples of this clearer perspective are cited in various parts of this paper. Implementations of systems to support the relational model are not discussed.

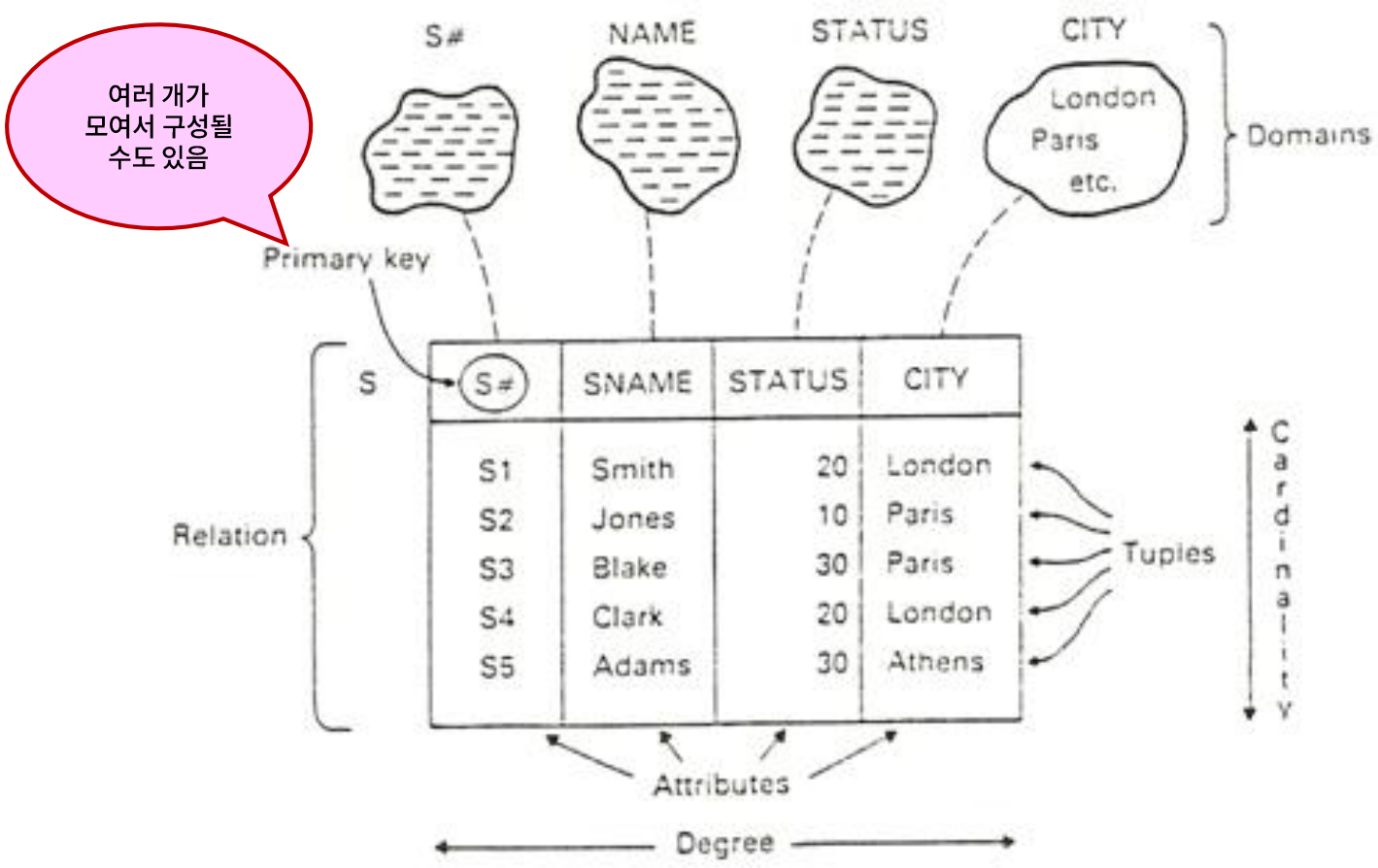
1.2. Data Dependencies in Present Systems
The provision of data description tables in recently developed information systems represents a major advance toward the goal of data independence [5, 6, 7]. Such tables facilitate changing certain characteristics of the data representation stored in a data bank. However, the variety of data representation characteristics which can be changed *without logically impairing some application programs* is still quite limited. Further, the model of data with which users interact is still cluttered with representational properties, particularly in regard to the representation of collections of data (as opposed to individual items). Three of the principal kinds of data dependencies which still need to be removed are: ordering dependence, indexing dependence, and access path dependence. In some systems these dependencies are not clearly separable from one another.

1.2.1. *Ordering Dependence.* Elements of data in a data bank may be stored in a variety of ways, some involving no concern for ordering, some permitting each element to participate in one ordering only, others permitting each element to participate in several orderings. Let us consider those existing systems which either require or permit data elements to be stored in at least one total ordering which is closely associated with the hardware-determined ordering of addresses. For example, the records of a file concerning parts might be stored in ascending order by part serial number. Such systems normally permit application programs to assume that the order of presentation of records from such a file is identical to (or is a subordering of) the

Volume 13 / Number 6 / June, 1970 Communications of the ACM 377

용어

- Relation (릴레이션)
  - 데이터가 저장되는 2차원 table
- Tuple (투플)
  - 릴레이션의 각 행 (row), record
- Attribute (애트리뷰트)
  - 릴레이션의 열 (column), field
- Cardinality (카디널리티)
  - 투플의 수
- Degree (차수)
  - 애트리뷰트의 수
- Primary key (프라이머리 키)
  - 테이블의 유일한 식별자
- Domain (도메인)
  - 애트리뷰트가 가질 수 있는 값의 집합



관계 데이터 모델 예제

학생 (STUDENT)
학번, 이름, 학년,
학과 정보
과목 (COURSE)
과목번호, 과목명, 학점,
담당교수
등록 (ENROL)
학생과 과목의 수강
관계 및 성적

학생 (STUDENT) Relation

릴레이션 스킴 (Relation Scheme) / 내포 (Intension)

Changes rarely (정적)

릴레이션 인스턴스 (Relation Instance) / 외연 (Extension)

Changes frequently (동적)

학번 (Sno)	이름 (Sname)	학년 (Year)	학과 (Dept)
100	나수영	4	컴퓨터
200	이찬수	3	전기
300	정기태	1	컴퓨터
400	송병길	4	컴퓨터
500	박종화	2	산공

릴레이션의 예제

INTEGER	CHAR(10)	INTEGER	CHAR(6)
DSNO	NAME	DYEAR	DEPT

도메인
제약조건, 타입

애트리뷰트

학생
(STUDENT)

학번 (Sno)	이름 (Sname)	학년 (Year)	학과 (Dept)
100	나 수 영	4	컴퓨터
200	이 찬 수	3	전기
300	정 기 태	1	컴퓨터
400	송 병 길	4	컴퓨터
500	박 종 화	2	산공

릴레이션 스킴

투플

인스턴스

학생 (STUDENT) 릴레이션

릴레이션 (Relation)

- '테이블'이라고 생각
  - 추상 객체의 구체적인 표현
  - 정확히 맞지는 않지만 직관적으로 이해하기 편리함
- 애트리뷰트 이름은 중복되면 안됨
  - 애트리뷰트 이름은 구분 가능해야 함
  - 같은 도메인을 갖는 애트리뷰트들 존재할 수 있음
    - 각각 구분되는 역할 담당

도메인

- 쪼개질 수 없음 (Atomic, 원자성)
  - 가장 작은 의미있는 단위 (≡scalar value)
  - 분해할 수 없음 (분해하면 의미를 잃음)
- 도메인의 부분 집합이 애트리뷰트임
  - 같은 타입을 갖는 스칼라 값들의 이름 붙여진 집합
  - 애트리뷰트가 취할 수 있는 값(value)들의 집합
  - 단순 도메인 (단순 애트리뷰트)
    - 단순 애트리뷰트 : 원자값
  - 복합 도메인(복합 애트리뷰트)
    - 단순 도메인의 조합
    - 복합 애트리뷰트 : 복합값
      - 연, 월, 일 ⇒ 날짜:<연,월,일>
    - Attribute 값을 의미상 하나의 단위로 취급하면 문제 없음

애트리뷰트 이름과 도메인 이름은 같을 수도 있음

예제)
나이: INT + 0세 이상
주문량: INT + 1개 이상

타입 + 제약

값의 집합이 되면 Unnormalized relation (비정규화 릴레이션)

값의 집합은 허용하지 않음

학생 도메인 정의 예제

SQL 학생 도메인 정의

타입 + 제약

```sql
/* 1. 학번 도메인: 8자리 숫자 문자열로 제한 */
CREATE DOMAIN StudentID AS CHAR(8)
CHECK (VALUE LIKE '[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]');

/* 2. 이름 도메인: 빈 값을 허용하지 않는 문자열 */
CREATE DOMAIN StudentName AS VARCHAR(20)
NOT NULL;

/* 3. 학년 도메인: 1학년부터 5학년까지만 허용 */
CREATE DOMAIN StudentYear AS INT
CHECK (VALUE >= 1 AND VALUE <= 5);

/* 4. 학부 도메인: 정해진 학과 이름 중 하나여야 함 (Enum 성격) */
CREATE DOMAIN DeptName AS VARCHAR(30)
CHECK (VALUE IN ('컴퓨터공학부', '전자공학부', '경영학부', '화학공학부'));
```

도메인 비교

S: Supplyer (공급자)
P: Part (부품)
SP: 어떤 공급자가 어떤 부품 공급하는지

- 도메인의 중요성
  - 비교

```
SELECT  P.*, SP.*
FROM    P, SP
WHERE P.P# = SP.P#
```

같은 도메인으로 정의됨
타당함

```
SELECT  P.*, SP.*
FROM    P, SP
WHERE P.WEIGHT = SP.QTY
```

다른 도메인으로 정의됨
타당하지 않음

  - 사용자의 실수 예방
  - 사용자에게 경고

애트리뷰트

- 애트리뷰트 이름
  - 해당 도메인의 이름을 그대로 사용할 수도 있고, 별도의 이름을 사용할 수도 있음
  - 임의의 도메인이 릴레이션에서 담당하고 있는 역할의 이름으로 지정 (SNAME)

SQL 공급자 테이블 정의 with NAME Domain

```sql
CREATE   DOMAIN   NAME  CHAR(20);
CREATE TABLE S
    (SNO CHAR(5) NOT NULL,
     SNAME   NAME  NOT NULL,
     CITY  CHAR(15)  NOT NULL,
     PRIMARY  KEY (SNO) );
```

릴레이션

- 도메인 D1, D2, ⋯, Dn (n = degree) 에서 정의된 릴레이션 (R)
  - Heading (테이블 위) : (relation) schema, scheme, intension (내포)
    - 애트리뷰트–도메인 쌍 (AD-pair)의 집합
    - {(A1:D1), (A2:D2), ⋯ , (An : Dn) }
    - 단, 모든 애트리뷰트 이름(Aj)은 서로 중복되지 않고 고유해야 함
      - { (Sno:DSNO), (Sname:NAME), (Year:DYEAR), (Dept:DEPT) }
  - Body (테이블 아래): (relation) instance, state (상태), extension (외연)
    - 시간에 따라 변하는 투플(Tuple)들의 집합
    - 각 투플은 애트리뷰트–값 쌍(AV-pair)의 집합
    - {(A1:Vi1), (A2:Vi2), ⋯ , (An:Vin) } i = 1, ⋯, m (m = cardinality)
      - { (Sno:100), (Sname:나수영), (Year:4), (Dept:컴퓨터) },
      - { (Sno:200), (Sname:이찬수), (Year:3), (Dept:전기) },
      - ⋯

릴레이션 스킴 (Relation Scheme)

내포 (본질):
꽃은 식물이다,
향기가 난다

- 릴레이션 내포 (relation intension) 또는 릴레이션 스키마라고도 함
  - 릴레이션 이름 + 애트리뷰트 이름
  - R(A1, A2, … , An), Ai ⇔ Di
  - R({A1, A2, … , An})
    - 애트리뷰트 이름의 집합

Attirbute

Domain

n: Degree

- 정적 성질
  - 시간에 무관
  - 릴레이션 타입

릴레이션 인스턴스 (Relation Instance)

외연 (대상):
꽃에 해당하는
사물은 장미,
튤립, 목련 등

- 릴레이션 외연 (relation extension)이라고도 함
- 릴레이션 R의 릴레이션 인스턴스
  - 어느 한 시점에 릴레이션 R이 포함하고 있는 투플들의 집합
  - $\{< V1, V2, \ldots, Vn >\}$ $\quad Vi \in Di$
  - 릴레이션의 내용, 상태

Value

Domain

- $\{(attr1=V1, attr2=V2, \cdots, attrn=Vn)\}$
- 동적 성질
  - 삽입, 삭제, 갱신으로 시간에 따라 변함
  - 릴레이션 값(보통 릴레이션)

릴레이션 (Relation) R

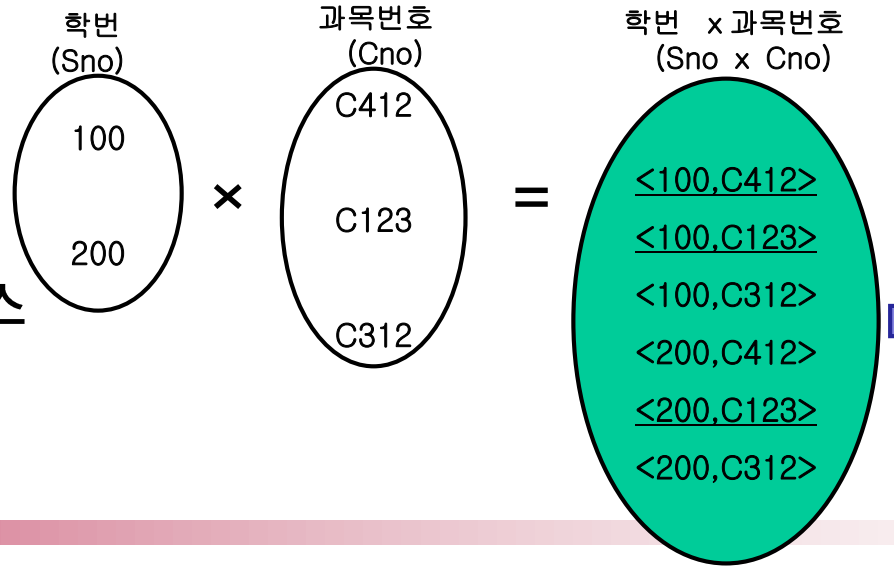
i . 수학적 정의

- 릴레이션 R : 카티션 프로덕트(Cartesian product)의 부분집합
- $R \subseteq D1 \times D2 \times \ldots \times Dn$
- 즉 n-투플 <d1, d2, … , dn>의 집합
- 단 Di : i번째 도메인 $di \in Di,\ i = 1,2, \ldots ,n$
- n : R의 차수(degree :일차, 이차, 삼차, … , n차)

Domain

ii . 개념적 정의

- 릴레이션 스킴 + 릴레이션 인스턴스



부분집합

학번 (Sno)	과목번호 (Cno)
100	C412
100	C123
200	C123

/3/26

릴레이션의 특성

매우 중요함

- 투플의 유일성
  - 중복된 투플이 없음
    - 릴레이션은 수학적인 집합
    - 프라이머리 키는 항상 존재함
  - 릴레이션 = 서로 다른 투플들의 "집합"
- 투플의 무순서성
  - 릴레이션 : 추상적 개념이고 투플들의 집합
    - 릴레이션은 수학적인 집합
  - 테이블 : 구체적 개념
- 애트리뷰트의 무순서성
  - 릴레이션 스킴 → 애트리뷰트들의 "집합"
  - Tuple : <attr:value>쌍의 집합
  - 애트리뷰트는 이름에 의해 참조됨
    - 순서(위치)에 의해 참조되지 않음
- 애트리뷰트의 원자성
  - 애트리뷰트 값 = 원자 값(atomic value)
    - 논리적으로 분해 불가능
  - 정규화 릴레이션 (normalized relation)
    - 비정규화 릴레이션은 분해로 정규화
    - 동등한 의미 유지
  - 널 값(null value) = 원자 값
    - unknown, inapplicable
  - 도메인
    - 단순 도메인
    - 복합 도메인 : 값을 하나의 단위로 취급

릴레이션의 Normalization

등록1
(ENROL1)

학번 (Sno)	과목성적 (Cgrade)	
	과목번호 (Cno)	성적 (Grade)
100	C413	A
	E412	A
200	C123	B
300	C312	A
	C324	C
	C413	A
400	C312	A
	C324	A
	C413	B
	E412	C
500	C312	B

(a) 비정규 릴레이션

등록
(ENROL)

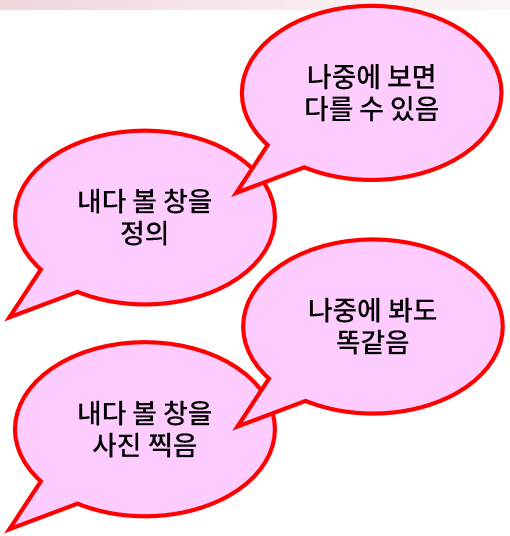
Degree
: 3

학번 (Sno)	과목번호 (Cno)	성적 (Grade)
100	C413	A
100	E412	A
200	C123	B
300	C312	A
300	C324	C
300	C413	A
400	C312	A
400	C324	A
400	C413	B
400	E412	C
500	C312	B

(b) 정규 릴레이션

릴레이션의 종류 (1/2)

- 종류
  - 베이스 릴레이션 (실제 릴레이션)
    - 데이터베이스에 저장된 이름 붙여진 릴레이션
  - 뷰 (View, 가상 릴레이션)
    - 이름 붙여진 파생 (derived) 릴레이션 (정의에 따라 만들어짐)
  - 스냅샷 (Snapshot)
    - 이름 붙여진 파생 릴레이션
    - 실제 값 저장, 가상 아님
    - PostgreSQL과 Oracle에서는 Materialized View라고 함
    - MySQL과 MariaDB에서는 Snapshot 지원 안함



Snapshot of Suppliers by City

```sql
CREATE MATERIALIZED VIEW SC
AS
SELECT SNO, CITY FROM S;
```

릴레이션의 종류 (2/2)

- 종류 (cont'd)
  - 질의 결과
    - 쿼리 실행의 결과로 생성되는 최종 출력 릴레이션
    - 데이터베이스 내에 영구적으로 존재하지 않고, 일시적으로 존재한 뒤에 사라짐
  - 중간 결과
    - 더 큰 질의 내에 중첩된 질의(sub-query 등)로부터 도출된 결과
    - 데이터베이스 내에 영구적으로 존재하지 않고, 최종 결과를 만들기 위해 잠시 존재한 뒤에 사라짐
    - 예제 SQL: 부품 'P2'를 취급하는 모든 공급업체의 거주 도시 목록을 중복 없이 출력해라.

```
SELECT DISTINCT S.CITY
FROM S
WHERE S.S# IN (SELECT SP.S#
               FROM SP
               WHERE SP.P# = 'P2');
```

  - 임시 릴레이션
    - 이름을 가진 릴레이션으로, 적절한 시점에 자동으로 삭제됨

관계형 (Relational) 데이터베이스

- 관계(형) 데이터베이스
  - 데이터베이스를 시간에 따라 그 내용(상태)이 변할 수 있는 테이블 형태로 표현
  - 사용자가, 시간에 따라 변하는 다양한 (assorted) degrees의 정규화된 릴레이션의 모음으로 인식하는 데이터베이스
- 관계 데이터베이스 스키마
  - = {릴레이션 스킴} + 무결성 제약조건
- 파일 시스템과의 비교
  - Relation : file
  - Tuple : record
    - occurrence: 현실 세계의 한 entity가 DB에 출현(발생)한 데이터 (실체, 사례)
  - Attribute : field

예제 (1/2)

- 대학(University) 관계 데이터베이스

학생 (STUDENT)

학번 (Sno)	이름 (Sname)	학년 (Year)	학과 (Dept)
100	나 수 영	4	컴퓨터
200	이 찬 수	3	전기
300	정 기 태	1	컴퓨터
400	송 병 길	4	컴퓨터
500	박 종 화	2	산공

과목 (COURSE)

과목번호 (Cno)	과목이름 (Cname)	학점 (Credit)	학과 (Dept)	담당교수 (PRname)
C123	프로그래밍	3	컴퓨터	김성국
C312	자료 구조	3	컴퓨터	황수관
C324	화일 구조	3	컴퓨터	이규찬
C413	데이터베이스	3	컴퓨터	이일로
E412	반 도 체	3	전자	홍봉진

예제 (2/2)

- 대학(University) 관계 데이터베이스

등록

(ENROL)

학번 (Sno)	과목번호 (Cno)	성적 (Grade)	중간성적 (Midterm)	기말성적 (Final)
100	C413	A	90	95
100	E412	A	95	95
200	C123	B	85	80
300	C312	A	90	95
300	C324	C	75	75
300	C413	A	95	90
400	C312	A	90	95
400	C324	A	95	90
400	C413	B	80	85
400	E412	C	65	75
500	C312	B	85	80